Project Report

Settlement Q&A Agent for Fintech Support

Problem Statement-8

Domain - Fintech / Payment Operations / Support

Tech Stack - FastAPI, PostgreSQL, SQLAlchemy, React, OpenAI API

Core Product - Deterministic investigation + AI-assisted explanation

1. Abstract :

Settlement Q&A Agent is a fintech support investigation system designed to answer a common operational question: why was a payment settlement not processed as expected? The application traces a transaction across mock payment gateway, bank, and internal ledger records, reconciles the available evidence, identifies discrepancies, and presents a likely root cause with an explicit confidence or uncertainty state.

The system separates investigation from language generation. A deterministic backend establishes the available facts first. An AI layer then converts that verified result into a concise explanation suitable for a support agent. This evidence-first design reduces hallucination risk and makes the investigation auditable.

---

2. Problem Statement :

Fintech support teams may receive questions such as “Why wasn't my settlement processed?” Answering these questions can require manual inspection of several systems, including the payment gateway, bank, and internal ledger. Their records can contain different statuses, timestamps, amounts, settlement identifiers, duplicate entries, or missing records.

The objective is to automate this investigation workflow and provide an evidence-based explanation without presenting unsupported conclusions as facts.

---

3. Objectives :

- Accept a transaction ID as the primary investigation input.
- Retrieve corresponding records from gateway, bank, and ledger sources.
- Reconcile statuses, amounts, settlement IDs, timestamps, and record presence.
- Detect missing, duplicate, mismatched, or conflicting evidence.
- Determine a likely settlement state and root cause using deterministic rules.
- Represent uncertainty when evidence is insufficient or contradictory.
- Use AI to explain the verified investigation result in plain English.
- Provide a support-oriented interface for fast transaction investigation.
- Keep evidence, system conclusions, and AI-generated explanation visibly distinct.

---

4. Proposed Solution :

The application is designed as an investigation assistant rather than a general-purpose chatbot. A support user enters a transaction ID. The backend traces it through the three data sources, builds a structured investigation result, and presents the findings. The AI component receives that structured result and produces a concise explanation.

4.1 Core Workflow :

- Transaction — Enter the transaction ID.
- Trace — Retrieve matching records from Gateway, Bank, and Ledger.
- Reconcile — Compare statuses, amounts, identifiers, and timestamps.
- Detect — Identify missing, duplicate, or conflicting evidence.
- Investigate — Determine settlement state, likely root cause, confidence, and recommended action.
- Explain — Convert the verified result into a support-friendly explanation using AI.

---

5. System Architecture :

The application follows a layered architecture so database access, deterministic investigation, AI explanation, and presentation remain separated.

Layer	Technology / Component	Responsibility
Frontend	React + Vite	Transaction investigation UI, results, timeline, evidence, demo cases
API	FastAPI	HTTP endpoints and request/response handling
Investigation	Python	Deterministic reconciliation and root-cause rules
Data Access	SQLAlchemy	Database models, sessions, and record retrieval
Database	PostgreSQL	Gateway, bank, and ledger records
AI Layer	OpenAI API	Natural-language explanation of verified investigation results

---

5.1 Logical Flow :

Support User → React UI → FastAPI → Investigation Engine → PostgreSQL
↓
Verified Result → AI Explanation

---

6. Data Design :

Generated mock data simulates the independent systems involved in payment settlement operations. A reproducible Python generator creates a canonical transaction set and intentionally injects investigation scenarios.

Source	Database Table	Purpose
Payment Gateway	gateway_records	Original transaction information and gateway status
Bank	bank_records	Settlement-side bank processing and posting information
Internal Ledger	ledger_records	Internal accounting/settlement state

6.1 Demonstration Scenarios :

Demo ID	Scenario	Investigation Signal
TXN-DEMO-001	Normal settlement	Complete and consistent evidence
TXN-DEMO-002	Bank delay	Gateway succeeded while bank processing remains pending
TXN-DEMO-003	Missing bank record	Bank evidence is absent
TXN-DEMO-004	Amount mismatch	Bank amount differs from gateway amount
TXN-DEMO-005	Missing ledger record	Ledger evidence is absent
TXN-DEMO-006	Settlement never initiated	Settlement-side records are absent
TXN-DEMO-007	Duplicate settlement	Multiple settlement-related records are present

7. Backend Implementation :

FastAPI provides the HTTP backend. SQLAlchemy provides the database abstraction and PostgreSQL stores the imported records. The data layer returns lists for each source, allowing missing and duplicate records to be represented explicitly.

The investigation layer consumes these records and produces a structured result containing evidence, discrepancies, settlement status, root-cause assessment, confidence, and recommended action.

7.1 API

Endpoint	Purpose
GET /health	Checks backend availability
GET /transactions/{transaction_id}	Retrieves gateway, bank, and ledger evidence
Investigation endpoint	Returns the deterministic investigation result used by UI and AI

---

8. Deterministic Investigation Engine :

The investigation engine is the core reasoning component. It applies explicit reconciliation rules rather than relying on the language model to determine transaction facts.

- Gateway success + bank pending → possible bank processing delay.
- Expected bank evidence absent → missing bank evidence or insufficient evidence, depending on context.
- Expected ledger evidence absent → missing ledger evidence.
- Gateway and bank amounts differ → amount mismatch.
- Multiple settlement records → duplicate settlement signal.
- No settlement-side evidence after a successful payment → settlement may not have been initiated.
- Conflicting evidence → mark the result uncertain and expose the conflict.

Design principle: Code determines what happened; AI explains what happened.

---

9. AI-Assisted Explanation :

The AI layer is downstream of the deterministic investigation. Instead of freely querying the database, the model receives the structured investigation result as context.

- Summarize what happened.
- Explain the likely reason for the settlement state.
- Reference the evidence supporting the conclusion.
- Suggest a reasonable support action.
- Clearly communicate uncertainty when evidence is incomplete or conflicting.
- Avoid inventing timestamps, amounts, IDs, statuses, or causes not present in the investigation result.

---

10. Frontend and User Experience :

The frontend is designed as an internal fintech support/investigation tool rather than a generic chatbot. The primary workflow starts with a transaction ID and presents the investigation result in a clear information hierarchy.

- Investigate — primary transaction lookup and investigation screen.
- How It Works — explains tracing, reconciliation, detection, investigation, and AI explanation.
- Demo Cases — quick access to predefined transaction IDs while still using the backend.
- Connected Systems — identifies the Gateway, Bank, and Ledger sources without pretending they are live production integrations.

The result view prioritizes settlement/root-cause status, evidence, discrepancies, confidence or uncertainty, timeline information, and recommended action. The AI explanation remains visually secondary.

---

11. Reliability and Edge Cases :

- Nonexistent transaction IDs should return an appropriate not-found response.
- Missing bank or ledger records are represented explicitly.
- Duplicate records remain visible so they can be detected.
- Conflicting evidence produces an uncertainty state instead of a fabricated conclusion.
- AI/API failure should not invalidate the underlying deterministic investigation.
- Demo data must not be presented as live financial-system data.
- API credentials and other secrets should be stored in environment variables.

---

12. Testing Strategy :

Testing focuses on the investigation path because correctness of tracing and reconciliation is the primary project requirement.

- Database retrieval and API tests.
- Normal transaction with gateway, bank, and ledger records.
- Missing bank record.
- Missing ledger record.
- Duplicate settlement records.
- Amount mismatch.
- Settlement not initiated.
- Nonexistent transaction.
- Insufficient or conflicting evidence.
- Frontend/backend integration and build checks.
- AI unavailable or missing API-key behavior.

---

13. Security and Responsible AI :

- The demonstration uses mock financial data rather than real customer/payment information.
- Secrets such as API keys should be stored in environment variables and excluded from source control.
- The AI layer is constrained to verified investigation output.
- The system communicates uncertainty instead of presenting unsupported guesses as facts.
- The mock Gateway, Bank, and Ledger must not be represented as live external systems.

---

14. Limitations :

- The demonstration uses mock CSV-derived data rather than live financial integrations.
- Reconciliation rules cover the implemented scenarios and are not a complete production settlement engine.
- AI explanations depend on the availability and behavior of the configured language model.
- Production use would require stronger authentication, authorization, audit logging, encryption, monitoring, and real integrations.

---

15. Future Enhancements :

- Integration with real payment gateway, bank, and ledger APIs.
- Role-based access for support, operations, and administrators.
- Persistent investigation history and audit logs.
- Expanded reconciliation rules and anomaly detection.
- Evidence-linked AI claims so every explanation can be traced to a source record.

- Batch investigation and operational dashboards.
- Production-grade observability, alerting, and access controls.

---

16. Conclusion :

Settlement Q&A Agent demonstrates how deterministic software engineering and AI can be combined for a high-value fintech support workflow. The system automates transaction tracing across multiple records, highlights discrepancies, determines a likely settlement outcome, and uses AI to communicate the result clearly. Its central principle is evidence first: the backend establishes the facts, while AI provides the explanation. This makes the system more transparent, testable, and appropriate for financial support scenarios.

Appendix — Suggested Demo Flow :

1. Open the investigation screen.
2. Enter TXN-DEMO-001 and show a normal settlement.
3. Enter TXN-DEMO-002 and demonstrate a bank delay.
4. Enter TXN-DEMO-004 and highlight an amount mismatch.
5. Enter TXN-DEMO-003 or TXN-DEMO-005 to demonstrate missing evidence.
6. Show the evidence/timeline section.
7. Show the AI-assisted explanation and explain that it is generated from the deterministic result.
8. Demonstrate uncertainty handling for incomplete or conflicting evidence.

Team Name : Zero/Zero
Team Members : Kartikey Srivastava(25BAI11235)
Atharva Kulkarni(25BAI10584)
Utkarsh(25BAI11231)